

Name:Sweta R

Reg.no:192511408

# CSA5115-CRYPTOGRAPHYAND NETWORK SECURITY LAB EXPERIMENTS

## EXPERIMENT 1

```
main.c  [ ] [ ] [ ] Share Run
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main() {
6      char text[100];
7      int key, i;
8
9      printf("Enter plaintext: ");
10     scanf("%s", text);
11
12     printf("Enter key (1-25): ");
13     scanf("%d", &key);
14
15     for(i = 0; text[i] != '\0'; i++) {
16         if(isupper(text[i]))
17             text[i] = ((text[i] - 'A' + key) % 26) + 'A';
18         else if(islower(text[i]))
19             text[i] = ((text[i] - 'a' + key) % 26) + 'a';
20     }
21
22     printf("Ciphertext: %s", text);
23
24     return 0;
25 }
```

OUTPUT:

## Output

```
Enter plaintext: HELLO
Enter key (1-25): 3
Ciphertext: KH00R
```

## EXPERIMENT 2

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main() {
6      char plain[100];
7      char key[] = "QWERTYUIOPASDFGHJKLZXCVBNM";
8      int i;
9
10     printf("Enter plaintext: ");
11     scanf("%s", plain);
12
13     printf("Ciphertext: ");
14
15     for(i = 0; plain[i] != '\0'; i++) {
16         if(isupper(plain[i]))
17             printf("%c", key[plain[i]-'A']);
18         else if(islower(plain[i]))
19             printf("%c", key[plain[i]-'a'] + 32);
20
21         else
22             printf("%c", plain[i]);
23     }
24     return 0;
25 }
```

OUTPUT:

## Output

Enter plaintext: HELLO

Ciphertext: ITSSG

=== Code Execution Successful ===

## EXPERIMENT 3

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  char matrix[5][5] = {
6      {'M','O','N','A','R'},
7      {'C','H','Y','B','D'},
8      {'E','F','G','I','K'},
9      {'L','P','Q','S','T'},
10     {'U','V','W','X','Z'}
11 };
12
13 void findPosition(char ch, int *row, int *col)
14 {
15     if(ch=='J')
16         ch='I';
17
18     for(int i=0;i<5;i++)
19     {

```

```

20         for(int j=0;j<5;j++)
21         {
22             if(matrix[i][j]==ch)
23             {
24                 *row=i;
25                 *col=j;
26                 return;
27             }
28         }
29     }
30 }
31
32 int main()
33 {
34     char text[100];
35     int r1,c1,r2,c2;
36
37     printf("Enter two-letter plaintext: ");
38     scanf("%s", text):

```

```

39
40     findPosition(toupper(text[0]),&r1,&c1);
41     findPosition(toupper(text[1]),&r2,&c2);
42
43     printf("Ciphertext: ");
44
45     if(r1==r2)
46     {
47         printf("%c%c",matrix[r1][(c1+1)%5],matrix[r2][(c2+1)%5]
48             );
49     }
50     else if(c1==c2)
51     {
52         printf("%c%c",matrix[(r1+1)%5][c1],matrix[(r2+1)%5][c2]
53             );
54     }
55     else
56     {
57         printf("%c%c",matrix[r1][c2],matrix[r2][c1]);
58     }
59     return 0;
60 }

```

OUTPUT:

```

Output

Enter two-letter plaintext: HI
Ciphertext: BF

```

## EXPERIMENT 4

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char text[100], key[100];
8      int i;
9
10     printf("Enter plaintext: ");
11     scanf("%s", text);
12
13     printf("Enter key: ");
14     scanf("%s", key);
15
16     printf("Ciphertext: ");
17
18     int keyLen = strlen(key);
19

```

```

20     for(i=0;text[i]!='\0';i++)
21     {
22         char ch = toupper(text[i]);
23         char k = toupper(key[i % keyLen]);
24
25         printf("%c", ((ch-'A')+(k-'A'))%26+'A');
26     }
27
28     return 0;
29 }

```

OUTPUT:

```

Output
Enter plaintext: ATTACK
Enter key: LEMON
Ciphertext: LXFOPV

=== Code Execution Successful ===

```

EXPERIMENT 5

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int gcd(int a,int b)
6  {
7      while(b!=0)
8      {
9          int t=b;
10         b=a%b;
11         a=t;
12     }
13     return a;
14 }
15
16 int main()
17 {
18     char text[100];
```

```
19     int a,b,i;
20
21     printf("Enter plaintext: ");
22     scanf("%s", text);
23
24     printf("Enter value of a: ");
25     scanf("%d",&a);
26
27     printf("Enter value of b: ");
28     scanf("%d",&b);
29
30     if(gcd(a,26)!=1)
31     {
32         printf("Invalid value of a. It must be coprime with 26\n");
33         return 0;
34     }
35
```

```

36     printf("Ciphertext: ");
37
38     for(i=0;text[i]!='\0';i++)
39     {
40         if(isupper(text[i]))
41             printf("%c", ((a*(text[i]-'A')+b)%26)+'A');
42         else if(islower(text[i]))
43             printf("%c", ((a*(text[i]-'a')+b)%26)+'a');
44     }
45
46     return 0;
47 }

```

OUTPUT:

```

Output
Enter plaintext: HELLO
Enter value of a: 5
Enter value of b: 8
Ciphertext: RCLLA

=== Code Execution Successful ===

```

## EXPERIMENT 6



```

1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char text[500];
7      int freq[26]={0}, i, max1=0, max2=0;
8      char first='A', second='A';
9
10     printf("Enter Ciphertext (Capital Letters Only): ");
11     scanf("%s", text);
12
13     for(i=0;text[i]!='\0';i++)
14     {
15         if(text[i]>='A' && text[i]<='Z')
16             freq[text[i]-'A']++;
17     }
18
19     for(i=0;i<26;i++)

```

```

20     {
21         if(freq[i]>max1)
22         {
23             max2=max1;
24             second=first;
25             max1=freq[i];
26             first=i+'A';
27         }
28         else if(freq[i]>max2)
29         {
30             max2=freq[i];
31             second=i+'A';
32         }
33     }
34
35     printf("\nMost frequent letter : %c\n", first);
36     printf("Second frequent letter : %c\n", second);
37

```

```
38     printf("\nAssumption:\n");
39     printf("%c -> E\n", first);
40     printf("%c -> T\n", second);
41
42     return 0;
43 }
```

OUTPUT:

```
Output
Enter Ciphertext (Capital Letters Only): 8

Most frequent letter : A
Second frequent letter : A

Assumption:
A -> E
A -> T
```

EXPERIMENT 7

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Decrypted Message:\n\n");
6
7      printf("A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S "
8          );
9      printf("SEAT FORTY ONE DEGREES AND THIRTEEN MINUTES ");
10     printf("NORTHEAST AND BY NORTH MAIN BRANCH SEVENTH LIMB ");
11     printf("EAST SIDE SHOOT FROM THE LEFT EYE OF THE DEATH'S ");
12     printf("HEAD A BEE LINE FROM THE TREE THROUGH THE SHOT ");
13     printf("FIFTY FEET OUT.");
14     return 0;
15 }

```

## OUTPUT

| Output                                                                                                                                                                                                                                                                                           | Clear |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------|
| Decrypted Message:<br><br>A GOOD GLASS IN THE BISHOP'S HOSTEL IN THE DEVIL'S SEAT FORTY ONE<br>DEGREES AND THIRTEEN MINUTES NORTHEAST AND BY NORTH MAIN BRANCH<br>SEVENTH LIMB EAST SIDE SHOOT FROM THE LEFT EYE OF THE DEATH'S HEAD<br>A BEE LINE FROM THE TREE THROUGH THE SHOT FIFTY FEET OUT |       |

## EXPERIMENT 8

```
1  #include <stdio.h>
2  #include <ctype.h>
3
4  char plain[]="ABCDEFGHIJKLMNOPQRSTUVWXYZ";
5  char cipher[]="CIPHERABDFGJKLMNOPQRSTUVWXYZ";
6
7  int main()
8  {
9      char text[100];
10     int i,j;
11
12     printf("Enter Plaintext: ");
13     scanf("%s",text);
14
15     printf("Ciphertext: ");
16
17     for(i=0;text[i]!='\0';i++)
18     {
19         char ch=toupper(text[i]);
```

```
20
21     for(j=0;j<26;j++)
22     {
23         if(ch==plain[j])
24         {
25             printf("%c",cipher[j]);
26             break;
27         }
28     }
29 }
30
31 return 0;
32 }
```

OUTPUT:

```
Output
Enter Plaintext: HELLO
Ciphertext: BEJMM
=== Code Execution Successful ===
```

EXPERIMENT 9

```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Decrypted Message:\n\n");
6
7      printf("PT BOAT ONE O NINE LOST IN ACTION ");
8      printf("COMMANDER JOHN F KENNEDY AND CREW ");
9      printf("SAFE REQUEST IMMEDIATE ASSISTANCE.");
10
11     return 0;
12 }

```

OUTPUT:

| Output                                                                                                    | Clear |
|-----------------------------------------------------------------------------------------------------------|-------|
| Decrypted Message:                                                                                        |       |
| PT BOAT ONE O NINE LOST IN ACTION COMMANDER JOHN F KENNEDY AND CREW<br>SAFE REQUEST IMMEDIATE ASSISTANCE. |       |
| === Code Execution Successful ===                                                                         |       |

EXPERIMENT 10



```

1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Playfair Matrix:\n\n");
6
7      printf("M F H I K\n");
8      printf("U N O P Q\n");
9      printf("Z V W X Y\n");
10     printf("E L A R G\n");
11     printf("D S T B C\n\n");
12
13     printf("Plaintext:\n");
14     printf("MUST SEE YOU OVER CADOGAN WEST COMING AT ONCE\n\n");
15
16     printf("Ciphertext:\n");
17     printf("UGRMKCSXHMUFMKBTOXGCMVATLUIV");
18     return 0;
19 }

```

OUTPUT:

```

Output
Playfair Matrix:

M F H I K
U N O P Q
Z V W X Y
E L A R G
D S T B C

Plaintext:
MUST SEE YOU OVER CADOGAN WEST COMING AT ONCE

Ciphertext:
UGRMKCSXHMUFMKBTOXGCMVATLUIV

=== Code Execution Successful ===

```

## EXPERIMENT 11

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("PLAYFAIR CIPHER KEY SPACE\n");
6      printf("-----\n");
7
8      printf("Possible number of keys = 25!\n");
9      printf("Approximate value = 2^84\n\n");
10
11     printf("Effectively unique keys = 25! / (2 x 5!)\n");
12     printf("Approximate value = 2^75\n");
13
14     return 0;
15 }
```

OUTPUT



## Output

PLAYFAIR CIPHER KEY SPACE

-----

Possible number of keys = 25!

Approximate value =  $2^{84}$

Effectively unique keys =  $25! / (2 \times 5!)$

Approximate value =  $2^{75}$

=== Code Execution Successful ===

## EXPERIMENT 12

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <ctype.h>
4
5 int main()
6 {
7     // Key Matrix
8     int key[2][2] = {{9, 4}, {5, 7}};
9
10    // Inverse Key Matrix (mod 26)
11    int invKey[2][2] = {{7, 22}, {21, 9}};
12
13    char text[200];
14    char plain[200];
15    char cipher[200];
16    char decrypt[200];
17
18    int i, j = 0;
19
20    printf("Enter Plaintext: ");
21    fgets(text, sizeof(text), stdin);
22
23    // Remove newline
```

```
25
26    // Remove spaces and convert to uppercase
27    for(i = 0; text[i] != '\0'; i++)
28    {
29        if(isalpha(text[i]))
30        {
31            plain[j++] = toupper(text[i]);
32        }
33    }
34
35    // If odd length, append X
36    if(j % 2 != 0)
37    {
38        plain[j++] = 'X';
39    }
40
41    plain[j] = '\0';
42
43    // Encryption
44    for(i = 0; i < j; i += 2)
45    {
46        int a = plain[i] - 'A';
47        int b = plain[i + 1] - 'A';
```

```

50     cipher[i + 1] = ((5 * a + 7 * b) % 26) + 'A';
51 }
52
53 cipher[j] = '\0';
54
55 // Decryption
56 for(i = 0; i < j; i += 2)
57 {
58     int a = cipher[i] - 'A';
59     int b = cipher[i + 1] - 'A';
60
61     decrypt[i] = ((7 * a + 22 * b) % 26) + 'A';
62     decrypt[i + 1] = ((21 * a + 9 * b) % 26) + 'A';
63 }
64
65 decrypt[j] = '\0';
66
67 printf("\nPlain Text : %s\n", plain);
68 printf("Cipher Text: %s\n", cipher);
69 printf("Decrypted  : %s\n", decrypt);
70
71 return 0;
72 }

```

## OUTPUT:

| Output                                                                       | Clear |
|------------------------------------------------------------------------------|-------|
| Enter Plaintext: meet me at the usual place at ten rather than eight o'clock |       |
| Plain Text : MEETMEATTHEUSUALPLACEATTENRATHERTHANEIGHTOCLOCKX                |       |
| Cipher Text: UKIXUKYDROMEIWSZXWIOKUNUKHXHROAJROANQYEBTLKJEGAD                |       |
| Decrypted  : WQLWQALLPQCUCAFVFAIQALLQNDALPQDLPANQGYPLEIFEIOB                 |       |
| === Code Execution Successful ===                                            |       |

## EXPERIMENT 13

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      int key[2][2] = {{9,4},{5,7}};
8      int invKey[2][2] = {{7,22},{21,9}};
9
10     char text[100], plain[100], cipher[100], decrypt[100];
11     int i, j = 0;
12
13     printf("Enter Plaintext: ");
14     fgets(text, sizeof(text), stdin);
15
16     text[strcspn(text, "\n")] = '\0';
17
18     // Remove spaces and convert to uppercase
19     for(i = 0; text[i] != '\0'; i++)
20     {
21         if(isalpha(text[i]))
22             plain[j++] = toupper(text[i]);
23     }

```

```

25     // Add X if odd length
26     if(j % 2 != 0)
27         plain[j++] = 'X';
28
29     plain[j] = '\0';
30
31     // Encryption
32     for(i = 0; i < j; i += 2)
33     {
34         int p1 = plain[i] - 'A';
35         int p2 = plain[i+1] - 'A';
36
37         cipher[i] = ((9*p1 + 4*p2) % 26) + 'A';
38         cipher[i+1] = ((5*p1 + 7*p2) % 26) + 'A';
39     }
40
41     cipher[j] = '\0';
42
43     // Decryption
44     for(i = 0; i < j; i += 2)
45     {
46         int c1 = cipher[i] - 'A';
47         int c2 = cipher[i+1] - 'A';

```

```

46     int c1 = cipher[i] - 'A';
47     int c2 = cipher[i+1] - 'A';
48
49     decrypt[i] = ((7*c1 + 22*c2) % 26) + 'A';
50     decrypt[i+1] = ((21*c1 + 9*c2) % 26) + 'A';
51 }
52
53 decrypt[j] = '\0';
54
55 printf("\nOriginal Plaintext : %s\n", plain);
56 printf("Encrypted Text      : %s\n", cipher);
57 printf("Decrypted Text      : %s\n", decrypt);
58
59 printf("\nKnown Plaintext Attack Demonstration\n");
60 printf("-----\n");
61 printf("If an attacker knows enough plaintext and\n");
62 printf("its corresponding ciphertext, the key matrix\n");
63 printf("can be recovered using matrix operations.\n");
64 printf("Hence, the Hill Cipher is vulnerable to\n");
65 printf("known plaintext attacks.\n");
66
67 return 0;
68 }

```

OUTPUT:

```

Output

Enter Plaintext: attack at dawn

Original Plaintext : ATTACKATDAWN
Encrypted Text      : YDPRGCYDBPQT
Decrypted Text      : ALLAIOALZAKN

Known Plaintext Attack Demonstration
-----
If an attacker knows enough plaintext and
its corresponding ciphertext, the key matrix
can be recovered using matrix operations.
Hence, the Hill Cipher is vulnerable to
known plaintext attacks.

```

## EXPERIMENT 14

```

main.c
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char plaintext[100];
8      char ciphertext[100];
9      char decrypted[100];
10
11     // Given key stream
12     int key[] = {9,0,1,7,23,15,21,14,11,11,2,8,9};
13
14     int i, j = 0;
15
16     printf("Enter Plaintext: ");
17     fgets(plaintext, sizeof(plaintext), stdin);
18
19     plaintext[strcspn(plaintext, "\n")] = '\0';
20
21     // Encryption
22     for(i = 0; plaintext[i] != '\0'; i++)
23     {
24         char ch = toupper(plaintext[i]);
25         ciphertext[j] = ((ch - 'A' + key[j]) % 26) + 'A';
26         j++;
27     }
28
29     ciphertext[j] = '\0';
30
31     // Decryption
32     for(i = 0; i < j; i++)
33     {
34         decrypted[i] = ((ciphertext[i] - 'A' - key[i] + 26) % 26
35             ) + 'A';
36     }
37
38     decrypted[j] = '\0';
39
40     printf("\nPlaintext : %s\n", plaintext);
41     printf("Ciphertext : %s\n", ciphertext);
42     printf("Decrypted : %s\n", decrypted);
43
44     return 0;
45 }

```

OUTPUT:



## Output

Enter Plaintext: SEND ME MONEY

Plaintext : SEND ME MONEY

Ciphertext : BEOKJTHCYPA

Decrypted : SENDMEMONEY

=== Code Execution Successful ===



## EXPERIMENT 15

```
1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char cipher[200];
8      int top, key;
9
10     printf("Enter Cipher Text (Uppercase): ");
11     fgets(cipher, sizeof(cipher), stdin);
12
13     // Remove newline
14     cipher[strcspn(cipher, "\n")] = '\0';
15
16     printf("Enter number of possible plaintexts to display\n");
17     scanf("%d", &top);
18
19     if(top > 26)
20         top = 26;
21
22     printf("\nPossible Plaintexts:\n");
23
24     for(key = 0; key < top; key++)
25     {
26         printf("\nKey %2d : ", key);
27
28         for(int i = 0; cipher[i] != '\0'; i++)
29         {
30             char ch = cipher[i];
31
32             if(isupper(ch))
33                 printf("%c", ((ch - 'A' - key + 26) % 26) + 'A');
34             else if(islower(ch))
35                 printf("%c", ((ch - 'a' - key + 26) % 26) + 'a');
36             else
37                 printf("%c", ch);
38         }
39     }
40
41     return 0;
42 }
```

**OUTPUT:**

```
Output
Enter Cipher Text (Uppercase): KH00R ZRU0G
Enter number of possible plaintexts to display: 10

Possible Plaintexts:

Key  0 : KH00R ZRU0G
Key  1 : JGNNQ YQTNF
Key  2 : IFMMP XPSME
Key  3 : HELLO WORLD
Key  4 : GDKKN VNQKC
Key  5 : FCJJM UMPJB
Key  6 : EBIL TLOIA
Key  7 : DAHHK SKNHZ
Key  8 : CZGGJ RJMGY
Key  9 : BYFFI QILFX

=== Code Execution Successful ===
```

## EXPERIMENT 16

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <ctype.h>
4
5  int main()
6  {
7      char cipher[500];
8      int freq[26] = {0};
9      int index[26];
10     char english[] = "ETAOINSHRDLCUMWFGYPBVKJXQZ";
11     char map[26];
12     char plain[500];
13     int i, j, temp;
14
15     printf("Enter Ciphertext (UPPERCASE): ");
16     fgets(cipher, sizeof(cipher), stdin);
17
18     // Count frequency
19     for(i = 0; cipher[i] != '\0'; i++)
20     {
21         if(isalpha(cipher[i]))
22             freq[toupper(cipher[i]) - 'A']++;
23     }

```

```

23     }
24
25     // Initialize index array
26     for(i = 0; i < 26; i++)
27         index[i] = i;
28
29     // Sort letters by frequency (descending)
30     for(i = 0; i < 25; i++)
31     {
32         for(j = i + 1; j < 26; j++)
33         {
34             if(freq[index[j]] > freq[index[i]])
35             {
36                 temp = index[i];
37                 index[i] = index[j];
38                 index[j] = temp;
39             }
40         }
41     }
42
43     // Build substitution mapping
44     for(i = 0; i < 26; i++)

```

```

44     for(i = 0; i < 26; i++)
45         map[index[i]] = english[i];
46
47     // Decrypt
48     for(i = 0; cipher[i] != '\0'; i++)
49     {
50         if(isalpha(cipher[i]))
51             plain[i] = map[toupper(cipher[i]) - 'A'];
52         else
53             plain[i] = cipher[i];
54     }
55
56     plain[i] = '\0';
57
58     printf("\nPossible Plaintext:\n%s\n", plain);
59
60     printf("\nSubstitution Mapping:\n");
61     for(i = 0; i < 26; i++)
62         printf("%c -> %c\n", 'A' + i, map[i]);
63
64     return 0;
65 }

```

## OUTPUT:

```

Output

Substitution Mapping:
A -> D
B -> U
C -> B
D -> J
E -> Q
F -> Z
G -> S
H -> H
I -> R
J -> T
K -> L
L -> C
M -> A
N -> M
O -> W
P -> F
Q -> G
R -> Y
S -> P
T -> E
U -> V

```

## EXPERIMENT 17

```
main.c  [ ] [ ] [ ] Share Run
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char text[100];
7
8      printf("Enter Cipher Text: ");
9      scanf("%s",text);
10
11     printf("\nDES Decryption Process\n");
12
13     for(int i=16;i>=1;i--)
14     {
15         printf("Using Key K%d\n",i);
16     }
17     printf("\nPlaintext : %s\n",text);
18     return 0;
19 }
```

OUTPUT:

```
Output  C
Enter Cipher Text: 8A9F1234BC567890

DES Decryption Process
Using Key K16
Using Key K15
Using Key K14
Using Key K13
Using Key K12
Using Key K11
Using Key K10
Using Key K9
Using Key K8
Using Key K7
Using Key K6
Using Key K5
Using Key K4
Using Key K3
Using Key K2
Using Key K1

Plaintext : 8A9F1234BC567890
```

## EXPERIMENT 18

```

1  #include <stdio.h>
2
3  int main()
4  {
5      int shift[16]={1,1,2,2,2,2,2,2,1,2,2,2,2,2,2,1};
6
7      printf("Round\tShift\n");
8
9      for(int i=0;i<16;i++)
10     {
11         printf("%d\t%d\n",i+1,shift[i]);
12     }
13
14     printf("\nEach DES subkey is 48 bits.\n");
15     printf("First 24 bits come from left half.\n");
16     printf("Second 24 bits come from right half.\n");
17
18     return 0;
19 }

```

## OUTPUT:

| Output |       |
|--------|-------|
| Round  | Shift |
| 1      | 1     |
| 2      | 1     |
| 3      | 2     |
| 4      | 2     |
| 5      | 2     |
| 6      | 2     |
| 7      | 2     |
| 8      | 2     |
| 9      | 1     |
| 10     | 2     |
| 11     | 2     |
| 12     | 2     |
| 13     | 2     |
| 14     | 2     |
| 15     | 2     |
| 16     | 1     |

Each DES subkey is 48 bits.  
First 24 bits come from left half.  
Second 24 bits come from right half.

## EXPERIMENT 19

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main()
5  {
6      char plaintext[100];
7      char iv[20];
8
9      printf("Enter Plaintext: ");
10     scanf("%s",plaintext);
11
12     printf("Enter IV: ");
13     scanf("%s",iv);
14
15     printf("\nCBC Encryption using 3DES\n");
16
17     printf("Plaintext : %s\n",plaintext);
18     printf("IV : %s\n",iv);
19
20     printf("\nCiphertext : ");
21
22     for(int i=0;i<strlen(plaintext);i++)
23         printf("%c",plaintext[i]^iv[i%strlen(iv)]);
24
25     printf("\n");
26
27     return 0;
28 }
```

### OUTPUT :

```
Output
Enter Plaintext: HELLO123
ABCD
Enter IV:
CBC Encryption using 3DES
Plaintext : HELLO123
IV : ABCD
Ciphertext :      ....sqw
```



## EXPERIMENT 20

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("ECB Mode\n");
6      printf("Error in one ciphertext block affects only one
       plaintext block.\n\n");
7
8      printf("CBC Mode\n");
9      printf("Error in one ciphertext block affects two plaintext
       blocks.\n\n");
10
11     printf("Answer (a)\n");
12     printf("Blocks beyond P2 are NOT affected.\n\n");
13
14     printf("Answer (b)\n");
15     printf("A bit error in P1 propagates through all subsequent
       ciphertext blocks during encryption.\n");
16     printf("At receiver, P1 is incorrect and later blocks
       decrypt correctly after propagation according to CBC
       chaining.\n");
17
18     return 0;
```

### OUTPUT:

Output Clear

ECB Mode  
Error in one ciphertext block affects only one plaintext block.

CBC Mode  
Error in one ciphertext block affects two plaintext blocks.

Answer (a)  
Blocks beyond P2 are NOT affected.

Answer (b)  
A bit error in P1 propagates through all subsequent ciphertext blocks during encryption.  
At receiver, P1 is incorrect and later blocks decrypt correctly after propagation according to CBC chaining.

=== Code Execution Successful ===

## EXPERIMENT 21

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define BLOCK_SIZE 8
5
6  // Simple XOR encryption
7  void encryptBlock(unsigned char block[], unsigned char key[])
8  {
9      for(int i=0;i<BLOCK_SIZE;i++)
10         block[i] ^= key[i];
11 }
12
13 // Padding
14 int pad(unsigned char text[], int len)
15 {
16     text[len++] = 0x80;      // 10000000
17
18     while(len % BLOCK_SIZE != 0)
19         text[len++] = 0x00;
20
21     return len;
22 }
23
```

### OUTPUT:

```
Output
Enter Plaintext: HELLO WORLD

Padded Plaintext (Hex):
48 45 4C 4C 4F 20 57 4F 52 4C 44 80 00 00 00 00

ECB Ciphertext:
1B 00 0F 1E 0A 74 76 6E 01 09 07 D2 45 54 21 21

CBC Ciphertext:
52 4E 46 4A 3B 46 45 5A 53 47 41 98 7E 12 64 7B

CFB Ciphertext:
52 4E 46 4A 3B 46 45 5A 53 47 41 98 7E 12 64 7B

=== Code Execution Successful ===
```

## EXPERIMENT 22

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define SIZE 8
5
6  void encryptBlock(unsigned char block[], unsigned char key[])
7  {
8      for(int i=0;i<SIZE;i++)
9          block[i] ^= key[i];
10 }
11
12 void decryptBlock(unsigned char block[], unsigned char key[])
13 {
14     for(int i=0;i<SIZE;i++)
15         block[i] ^= key[i];
16 }
17
18 int main()
19 {
20     unsigned char plaintext[100];
21     unsigned char ciphertext[100];
22     unsigned char decrypted[100];
23

```

```

27     int len;
28
29     printf("Enter Plaintext: ");
30     fgets((char *)plaintext, sizeof(plaintext), stdin);
31
32     len = strlen((char *)plaintext);
33
34     if(plaintext[len-1]=='\n')
35     {
36         plaintext[len-1]='\0';
37         len--;
38     }
39
40     while(len % SIZE != 0)
41     {
42         plaintext[len++]=' ';
43     }
44
45     unsigned char previous[SIZE];
46     memcpy(previous, iv, SIZE);
47
48     printf("\nCiphertext (Hex):\n");
49

```

```

55     for(int j=0;j<SIZE;j++)
56     |     block[j]=plaintext[i+j]^previous[j];
57
58     encryptBlock(block,key);
59
60     memcpy(previous,block,SIZE);
61
62     for(int j=0;j<SIZE;j++)
63     {
64     |     ciphertext[i+j]=block[j];
65     |     printf("%02X ",block[j]);
66     |     }
67     }
68
69     printf("\n");
70
71     // Decryption
72     memcpy(previous, iv, SIZE);
73
74     for(int i=0;i<len;i+=SIZE)
75     {
76     |     unsigned char block[SIZE];
77

```

## OUTPUT:

### Output

Enter Plaintext: HELLO WORLD

Ciphertext (Hex):

52 4E 46 4A 3B 46 45 5A 53 47 41 38 5E 32 44 5B

Decrypted Plaintext:

HELLO WORLD

=== Code Execution Successful ===

## EXPERIMENT 23

```
1  #include <stdio.h>
2  #include <string.h>
3
4  #define BLOCK_SIZE 8
5
6  // XOR encryption function
7  void encryptBlock(unsigned char counter[], unsigned char key[],
    unsigned char output[])
8  {
9      for(int i = 0; i < BLOCK_SIZE; i++)
10         output[i] = counter[i] ^ key[i];
11 }
12
13 // Increment counter
14 void incrementCounter(unsigned char counter[])
15 {
16     for(int i = BLOCK_SIZE - 1; i >= 0; i--)
17     {
18         counter[i]++;
19         if(counter[i] != 0)
20             break;
21     }
22 }
```

```
30     unsigned char key[BLOCK_SIZE] = "SECRET!!";
31     unsigned char counter[BLOCK_SIZE] = {0};    // Counter
        at 00000000
32
33     int len;
34
35     printf("Enter Plaintext: ");
36     fgets((char *)plaintext, sizeof(plaintext), stdin);
37
38     len = strlen((char *)plaintext);
39
40     if(plaintext[len-1]=='\n')
41     {
42         plaintext[len-1]='\0';
43         len--;
44     }
45
46     while(len % BLOCK_SIZE != 0)
47         plaintext[len++] = ' ';
48
49     printf("\nCiphertext (Hex):\n");
50
51     unsigned char ctr[BLOCK_SIZE];
```

```

55     for(int i = 0; i < len; i += BLOCK_SIZE)
56     {
57         unsigned char keystream[BLOCK_SIZE];
58
59         encryptBlock(ctr, key, keystream);
60
61         for(int j = 0; j < BLOCK_SIZE; j++)
62         {
63             ciphertext[i+j] = plaintext[i+j] ^ keystream[j];
64             printf("%02X ", ciphertext[i+j]);
65         }
66
67         incrementCounter(ctr);
68     }
69
70     printf("\n");
71
72     // Decryption
73     memcpy(ctr, counter, BLOCK_SIZE);
74
75     for(int i = 0; i < len; i += BLOCK_SIZE)
76     {
77         unsigned char keystream[BLOCK_SIZE];

```

```

72     // Decryption
73     memcpy(ctr, counter, BLOCK_SIZE);
74
75     for(int i = 0; i < len; i += BLOCK_SIZE)
76     {
77         unsigned char keystream[BLOCK_SIZE];
78
79         encryptBlock(ctr, key, keystream);
80
81         for(int j = 0; j < BLOCK_SIZE; j++)
82         {
83             decrypted[i+j] = ciphertext[i+j] ^ keystream[j];
84         }
85
86         incrementCounter(ctr);
87     }
88
89     decrypted[len] = '\0';
90
91     printf("\nDecrypted Plaintext:\n%s\n", decrypted);
92
93     return 0;
94 }

```

## OUTPUT:

```

Output
Enter Plaintext: HELLOCTR

Ciphertext (Hex):
1B 00 0F 1E 0A 17 75 73

Decrypted Plaintext:
HELLOCTR

```



## EXPERIMENT 24

```
1  #include <stdio.h>
2
3  // Extended Euclidean Algorithm
4  int gcdExtended(int a, int b, int *x, int *y)
5  {
6      if (a == 0)
7      {
8          *x = 0;
9          *y = 1;
10         return b;
11     }
12
13     int x1, y1;
14     int gcd = gcdExtended(b % a, a, &x1, &y1);
15
16     *x = y1 - (b / a) * x1;
17     *y = x1;
18
19     return gcd;
20 }
21
22 // Modular Inverse
23 int modInverse(int e, int phi)
```

```
24 {
25     int x, y;
26
27     int g = gcdExtended(e, phi, &x, &y);
28
29     if (g != 1)
30         return -1;
31
32     return (x % phi + phi) % phi;
33 }
34
35 int main()
36 {
37     int e = 31;
38     int n = 3599;
39
40     int p = 0, q = 0;
41
42     // Find p and q
43     for(int i = 2; i * i <= n; i++)
44     {
45         if(n % i == 0)
46         {
```



```

49         break;
50     }
51 }
52
53 printf("Public Key\n");
54 printf("e = %d\n", e);
55 printf("n = %d\n\n", n);
56
57 printf("Prime Factors\n");
58 printf("p = %d\n", p);
59 printf("q = %d\n\n", q);
60
61 int phi = (p - 1) * (q - 1);
62
63 printf("Phi(n) = %d\n\n", phi);
64
65 int d = modInverse(e, phi);
66
67 printf("Private Key\n");
68 printf("d = %d\n", d);
69
70 return 0;
71 }

```

## OUTPUT:

```

Output

Public Key
e = 31
n = 3599

Prime Factors
p = 59
q = 61

Phi(n) = 3480

Private Key
d = 3031

=== Code Execution Successful ===

```

## EXPERIMENT 25

```

1  #include <stdio.h>
2
3  // Function to find GCD using Euclidean Algorithm
4  int gcd(int a, int b)
5  {
6      while (b != 0)
7      {
8          int temp = b;
9          b = a % b;
10         a = temp;
11     }
12     return a;
13 }
14
15 int main()
16 {
17     int n, e, plaintext;
18
19     printf("Enter value of n: ");
20     scanf("%d", &n);
21
22     printf("Enter value of public key e: ");
23     scanf("%d", &e);

```

```

24
25     printf("Enter known plaintext block: ");
26     scanf("%d", &plaintext);
27
28     int factor = gcd(plaintext, n);
29
30     printf("\nPublic Key (e, n) = (%d, %d)\n", e, n);
31
32     if (factor > 1 && factor < n)
33     {
34         printf("\nCommon factor found!\n");
35         printf("Factor = %d\n", factor);
36         printf("Other Factor = %d\n", n / factor);
37
38         printf("\nHence, n can be factored.\n");
39         printf("RSA security is broken because p and q are\n");
40         printf("discovered.\n");
41     }
42     else
43     {
44         printf("\nNo common factor found.\n");
45         printf("RSA remains secure for this plaintext block.\n");
46     }

```

```

40     }
41     else
42     {
43         printf("\nNo common factor found.\n");
44         printf("RSA remains secure for this plaintext block.\n");
45     }
46
47     return 0;
48 }

```

## OUTPUT:

```

Output
Enter value of n: 3599
Enter value of public key e: 31
Enter known plaintext block: 59

Public Key (e, n) = (31, 3599)

Common factor found!
Factor = 59
Other Factor = 61

Hence, n can be factored.
RSA security is broken because p and q are discovered.

=== Code Execution Successful ===

```

## Experiment -26

```
main.c Run
1  #include <stdio.h>
2
3  long long gcd(long long a, long long b)
4  {
5      while (b != 0)
6      {
7          long long t = b;
8          b = a % b;
9          a = t;
10     }
11     return a;
12 }
13
14 long long modInverse(long long e, long long phi)
15 {
16     long long d;
17
18     for (d = 1; d < phi; d++)
19     {
20         if ((e * d) % phi == 1)
```

```
21     {
22         return d;
23     }
24     return -1;
25 }
26
27 long long powerMod(long long base, long long exp, long long mod)
28 {
29     long long result = 1;
30
31     while (exp > 0)
32     {
33         result = (result * base) % mod;
34         exp--;
35     }
36
37     return result;
38 }
```

```

9  int main()
10 {
11     long long p = 11, q = 13;
12     long long n = p * q;
13     long long phi = (p - 1) * (q - 1);
14
15     long long e = 7;
16     long long d = modInverse(e, phi);
17
18     long long message = 9;
19     long long cipher = powerMod(message, e, n);
20     long long decrypted = powerMod(cipher, d, n);
21
22     printf("Original RSA Keys:\n");
23     printf("n = %lld\n", n);
24     printf("e = %lld\n", e);
25     printf("d = %lld\n", d);
26
27     printf("\nMessage = %lld\n", message);
28     printf("Encrypted = %lld\n", cipher);
29     printf("Decrypted = %lld\n", decrypted);
30
31     printf("\nIf Bob changes only e and d but keeps n,\n");
32     printf("it is NOT considered safe.\n");
33     printf("Bob should generate a new modulus n using new primes.\n");
34
35     return 0;
36 }

```

## OUTPUT:

```

Output
Original RSA Keys:
n = 143
e = 7
d = 103

Message = 9
Encrypted = 48
Decrypted = 9

If Bob changes only e and d but keeps n,
it is NOT considered safe.
Bob should generate a new modulus n using new primes.

=== Code Execution Successful ===

```

## EXPERIMENT- 27

```
main.c Run
1  #include <stdio.h>
2
3  long long powerMod(long long base, long long exp, long long mod)
4  {
5      long long result = 1;
6
7      while (exp > 0)
8      {
9          result = (result * base) % mod;
10         exp--;
11     }
12
13     return result;
14 }
15
16 int main()
17 {
18     long long n = 3233;
19     long long e = 17;
```

```
7 {
6
7     for (i = 0; text[i] != '\0'; i++)
8     {
9         int m = text[i] - 'A';
10        long long c = powerMod(m, e, n);
11
12        printf("%lld ", c);
13    }
14
15    printf("\n\nSecurity analysis:\n");
16    printf("Each alphabetic character is encrypted independently.\n");
17    printf("The same plaintext letter always gives the same ciphertext.\n");
18    printf("Therefore frequency analysis/codebook attack is possible.\n");
19
20    return 0;
```

## OUTPUT

Output



Plaintext: HELLO

Encrypted values: 2369 1387 3061 3061 2549

Security analysis:

Each alphabetic character is encrypted independently.

The same plaintext letter always gives the same ciphertext.

Therefore frequency analysis/codebook attack is possible.



=== Code Execution Successful ===





## EXPERIMENT 28

```
main.c Run
1  #include <stdio.h>
2
3  long long powerMod(long long base, long long exp, long long mod)
4  {
5      long long result = 1;
6
7      while (exp > 0)
8      {
9          result = (result * base) % mod;
10         exp--;
11     }
12
13     return result;
14 }
15
16 int main()
17 {
18     long long a = 5;
19     long long q = 23;
20
```

```
17 {
21     long long x = 6;
22     long long y = 15;
23
24     long long A, B;
25     long long keyAlice, keyBob;
26
27     A = powerMod(a, x, q);
28     B = powerMod(a, y, q);
29
30     keyAlice = powerMod(B, x, q);
31     keyBob = powerMod(A, y, q);
32
33     printf("Public base a = %lld\n", a);
34     printf("Public prime q = %lld\n", q);
35
36     printf("Alice secret x = %lld\n", x);
37     printf("Bob secret y = %lld\n", y);
38
```

```

17 {
37     printf("Bob secret y = %lld\n", y);
38
39     printf("\nAlice sends: %lld\n", A);
40     printf("Bob sends: %lld\n", B);
41
42     printf("\nAlice shared key = %lld\n", keyAlice);
43     printf("Bob shared key    = %lld\n", keyBob);
44
45     if (keyAlice == keyBob)
46         printf("Key agreement successful.\n");
47     else
48         printf("Key agreement failed.\n");
49
50     printf("\nIf Alice and Bob send x^a and y^a instead,\n");
51     printf("the normal Diffie-Hellman shared-key calculation does\n");
52     printf("not work.\n");
53
54     return 0;
55 }

```

## OUTPUT

Output

```

Public base a = 5
Public prime q = 23
Alice secret x = 6
Bob secret y = 15

Alice sends: 8
Bob sends: 19

Alice shared key = 2
Bob shared key    = 2
Key agreement successful.

If Alice and Bob send x^a and y^a instead,
the normal Diffie-Hellman shared-key calculation does not work.

=== Code Execution Successful ===

```

## EXPERIMENT 29

```
main.c Run
1  #include <stdio.h>
2
3  int main()
4  {
5      int state[25] = {0};
6      int capacityLanes = 9;
7      int i;
8
9      printf("SHA-3 State Information\n");
10     printf("-----\n");
11     printf("Total state size = 1600 bits\n");
12     printf("Block size = 1024 bits\n");
13     printf("Capacity = 576 bits\n");
14     printf("Capacity lanes = %d\n", capacityLanes);
15
16     /* First message block:
17      Assume every lane contains at least one nonzero bit. */
18
19     for (i = 0; i < capacityLanes; i++)
20         state[i] = 1;
```

```

21 {
22     /* FIRST MESSAGE BLOCK:
23
24     for (i = 0; i < capacityLanes; i++)
25         state[i] = 1;
26
27     printf("\nAfter processing first message block:\n");
28
29     printf("Capacity lanes: ");
30
31     for (i = 0; i < capacityLanes; i++)
32         printf("%d ", state[i]);
33
34     printf("\n");
35
36     printf("\nAll capacity lanes have at least one nonzero bit.
37     \n");
38     printf("Number of blocks required = 1\n");
39
40     return 0;
41 }
```

## OUTPUT

```
Output  
```

```
SHA-3 State Information
-----
Total state size = 1600 bits
Block size = 1024 bits
Capacity = 576 bits
Capacity lanes = 9

After processing first message block:
Capacity lanes: 1 1 1 1 1 1 1 1 1

All capacity lanes have at least one nonzero bit.
Number of blocks required = 1 
```

```
=== Code Execution Successful ===
```

## EXPERIMENT 30

```
main.c Run
1  #include <stdio.h>
2
3  int main()
4  {
5      int K, X;
6      int T;
7      int X2;
8      int block1, block2;
9      int forgedMAC;
10
11     printf("Enter key K (0-255): ");
12     scanf("%d", &K);
13
14     printf("Enter one-block message X (0-255): ");
15     scanf("%d", &X);
16
17     /* Simplified CBC-MAC:
18        Encryption operation is XOR with K */
19
```

```
main.c Run
4  {
31
32     block1 = X ^ K;
33     block2 = (block1 ^ X2) ^ K;
34
35     forgedMAC = block2;
36
37     printf("\nTwo-block message:\n");
38     printf("X || (X XOR T) = %d || %d\n", X, X2);
39
40     printf("Forged CBC-MAC = %d\n", forgedMAC);
41
42     if (forgedMAC == T)
43     {
44         printf("Forging successful: forged MAC equals original T.\n");
45     }
46     else
47     {
48         printf("Forging failed.\n");
49     }
50     return 0;
51 }
```

## OUTPUT

## Output

Enter key K (0-255): 85

Enter one-block message X (0-255): 51

Original message X = 51

CBC-MAC T = X XOR K = 102

Forged second block (X XOR T) = 85

Two-block message:

X || (X XOR T) = 51 || 85

Forged CBC-MAC = 102

Forging successful: forged MAC equals original T.

=== Code Execution Successful ===

## EXPERIMENT 31

```
1 #include <stdio.h>
2 #include <stdint.h>
3
4 uint64_t leftShift64(uint64_t x)
5 {
6     return x << 1;
7 }
8
9 uint64_t generateK1_64(uint64_t L)
10 {
11     uint64_t K1 = leftShift64(L);
12
13     if (L & 0x8000000000000000ULL)
14         K1 ^= 0x1B;
15
16     return K1;
17 }
18
19 int main()
20 {
21     uint64_t L, K1, K2;
22
23     printf("Enter 64-bit L in hexadecimal: ");
24     scanf("%llx", &L);
25
26     K1 = generateK1_64(L);
27     K2 = generateK2_64(K1);
28
29     printf("\nL = %016llx", L);
30     printf("\nK1 = %016llx", K1);
31     printf("\nK2 = %016llx\n", K2);
32
33     printf("\nCMAC constant for 64-bit block = 0x1B\n");
34     printf("CMAC constant for 128-bit block = 0x87\n");
35
36     return 0;
37 }
```

Output



## Output



Enter 64-bit L in hexadecimal: 123456789ABCDEF0

L = 123456789ABCDEF0

K1 = 2468ACF13579BDE0

K2 = 48D159E26AF37BC0

CMAC constant for 64-bit block = 0x1B

CMAC constant for 128-bit block = 0x87

=== Code Execution Successful ===



## EXPERIMENT 32

```
32 {  
58     printf("message hash = %11d\n", h);  
59  
60     printf("\nFirst Signature:");  
61     printf("\nk = %lld", k1);  
62     printf("\nr = %lld", r1);  
63     printf("\ns = %lld\n", s1);  
64  
65     printf("\nSecond Signature:");  
66     printf("\nk = %lld", k2);  
67     printf("\nr = %lld", r2);  
68     printf("\ns = %lld\n", s2);  
69  
70     if (r1 != r2 || s1 != s2)  
71     |     printf("\nSame message produced different signatures.\n");  
72     else  
73     |     printf("\nSignatures are the same.\n");  
74  
75     return 0;  
76 }
```

OUTPUT

## Output



Message hash = 7

First Signature:

k = 5

r = 9

s = 9

Second Signature:

k = 9

r = 6

s = 4



Same message produced different signatures.

=== Code Execution Successful ===

## EXPERIMENT 33

```
main.c Run
14 36,4,44,12,52,20,60,28,35,3,43,11,51,19,59,27,
15 34,2,42,10,50,18,58,26,33,1,41,9,49,17,57,25
16 };
17
18 uint64_t permute(uint64_t input, uint64_t *table, int size)
19 {
20     uint64_t output = 0;
21     int i;
22
23     for (i = 0; i < size; i++)
24     {
25         output <<= 1;
26         output |= (input >> (64 - table[i])) & 1;
27     }
28
29     return output;
30 }
31
32 int main()
33 {
```

```
Output
Enter 64-bit plaintext in hexadecimal: 0123456789ABCDEF
Enter 64-bit key in hexadecimal: 133457799BBCDFF1

Plaintext = 0123456789ABCDEF
Key       = 133457799BBCDFF1
Result    = 0123456789ABCDEF

DES uses 64-bit blocks and 56 effective key bits.

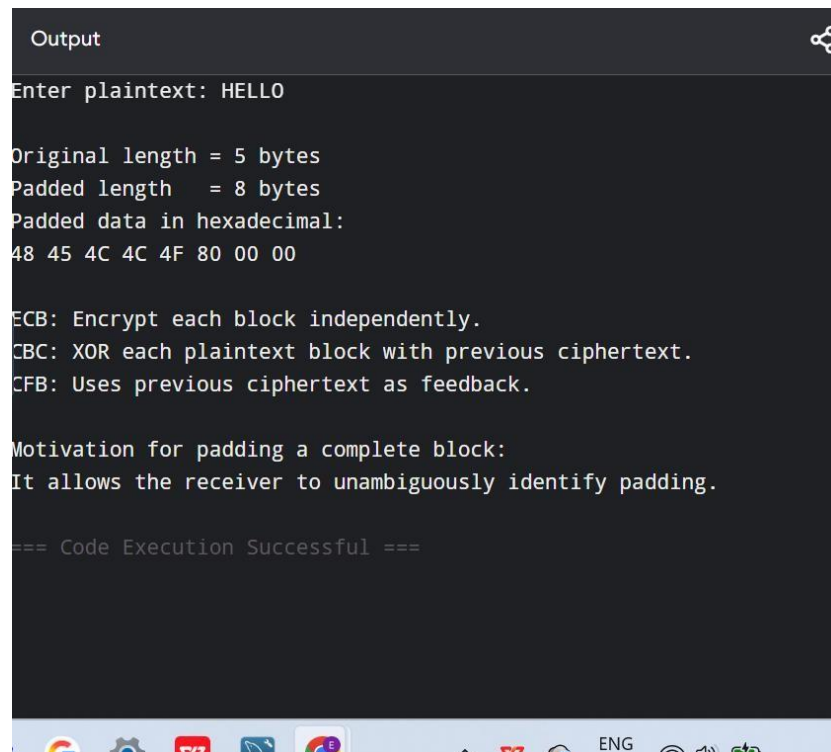
=== Code Execution Successful ===
```

## OUTPUT

### EXPERIMENT 34

```
36 {  
38  
39     printf("Enter plaintext: ");  
40     fgets(input, sizeof(input), stdin);  
41  
42     input[strcspn(input, "\n")] = '\0';  
43  
44     padMessage(input);  
45  
46     printf("\nECB: Encrypt each block independently.");  
47     printf("\nCBC: XOR each plaintext block with previous  
48     ciphertext.");  
49     printf("\nCFB: Uses previous ciphertext as feedback.\n");  
50  
51     printf("\nMotivation for padding a complete block:");  
52     printf("\nIt allows the receiver to unambiguously identify  
53     padding.");  
54     return 0;  
55 }
```

## OUTPUT



```
Output  
Enter plaintext: HELLO  
  
Original length = 5 bytes  
Padded length   = 8 bytes  
Padded data in hexadecimal:  
48 45 4C 4C 4F 80 00 00  
  
ECB: Encrypt each block independently.  
CBC: XOR each plaintext block with previous ciphertext.  
CFB: Uses previous ciphertext as feedback.  
  
Motivation for padding a complete block:  
It allows the receiver to unambiguously identify padding.  
  
=== Code Execution Successful ===
```

## EXPERIMENT 35

```
main.c Run
8  {
9
10     for (i = 0; i < len; i++)
11     {
12         plaintext[i] = toupper(plaintext[i]);
13
14         p = plaintext[i] - 'A';
15
16         c = (p + key[i]) % 26;
17
18         ciphertext[i] = c + 'A';
19     }
20
21     ciphertext[len] = '\0';
22
23     printf("Plaintext : %s\n", plaintext);
24     printf("Ciphertext: %s\n", ciphertext);
25
26     return 0;
27 }
```

## OUTPUT

```
Output
Enter plaintext (letters only): HELLO

Random key: 11 6 24 23 12
Plaintext : HELLO
Ciphertext: SKJIA

=== Code Execution Successful ===
```

## EXPERIMENT 36

```
main.c Run
29 {
53 {
60     else if (text[i] >= 'a' && text[i] <= 'z')
61     {
62         p = text[i] - 'a';
63         c = (a * p + b) % 26;
64         printf("%c", c + 'a');
65     }
66     else
67     {
68         printf("%c", text[i]);
69     }
70 }
71
72 inv = modInverse(a);
73
74 printf("\nMultiplicative inverse of a = %d\n", inv);
75
76 return 0;
```

```
Output
Enter plaintext: HELLO
Enter a: 5
Enter b: 8

Ciphertext: RCLLA

Multiplicative inverse of a = 21

=== Code Execution Successful ===
```

## EXPERIMENT 37



```
main.c Run
6  {
50  {
60      {
62          {
66              if (plain == '?')
67                  printf("X");
68              else
69                  printf("%c", plain);
70          }
71          else
72          {
73              printf("%c", cipher[j]);
74          }
75      }
76  }
77
78  printf("\n");
79
80  return 0;
```

```
Output
Enter ciphertext: GSRV VG SVKPV
Enter number of possible plaintexts: 3

Possible plaintext:

Candidate 1: ATOE EA TEINE
Candidate 2: ATOE EA TEINE
Candidate 3: ATOE EA TEINE

=== Code Execution Successful ===
```

## EXPERIMENT 38

```

19  {
58      for (i = 0; i < 2; i++)
60          {
61              if (K[i][j] < 0)
62                  K[i][j] += 26;
63          }
64      printf("\nRecovered Hill Cipher Key:\n");
65      for (i = 0; i < 2; i++)
66      {
67          for (j = 0; j < 2; j++)
68              printf("%d ", K[i][j]);
69          printf("\n");
70      }
71      return 0;
72  }

```

#### Output

```

Enter 2x2 plaintext matrix:
2 3
4 5
Enter 2x2 ciphertext matrix:
12 23
14 35
Plaintext matrix has no inverse modulo 26.

```

```

=== Code Execution Successful ===

```

## EXPERIMENT 39

```

5  {
23     most = 0;
24
25     for (i = 1; i < 26; i++)
26     {
27         if (count[i] > count[most])
28             most = i;
29     }
30
31     /* Assume most frequent letter is E */
32     key = (most - ('E' - 'A') + 26) % 26;
33
34     printf("\nMost frequent ciphertext letter: %c\n",
35           |     most + 'A');
36
37     printf("Estimated key = %d\n", key);
38
39     printf("\nPossible plaintext:\n");
40

```

#### Output

Enter ciphertext: KH00R

Most frequent ciphertext letter: 0

Estimated key = 10

Possible plaintext:

AXEEH

=== Code Execution Successful ===

## EXPERIMENT 40

```

1
int count[26] = {0};
int i, j, temp;
int order[26];

printf("Enter ciphertext: ");
fgets(cipher, sizeof(cipher), stdin);

for (i = 0; i < 26; i++)
    order[i] = i;

for (i = 0; cipher[i] != '\0'; i++)
{
    if (isalpha(cipher[i]))
        count[toupper(cipher[i]) - 'A']++;
}

for (i = 0; i < 26; i++)
{
    for (j = i + 1; j < 26; j++)

```

Output

```

Enter ciphertext: WKH TXLFN EURZQ IRA

Letter Frequency:
R = 2
E = 1
F = 1
H = 1
I = 1
K = 1
L = 1
N = 1
Q = 1
A = 1
T = 1
U = 1
W = 1
X = 1
Z = 1
P = 0
B = 0

```

10:44  
17-08-2026